IV - Imágenes

Computación gráfica y ambientes virtuales (0494)

Facultad de Ingeniería
Centro Regional Universitario Córdoba - IUA
Universidad de la Defensa Nacional

2026





Imágenes en computación gráfica

La imagen como resultado y como dato

Todos los métodos de visualización vistos en la materia terminan en una imagen: el *pipeline* gráfico la deja en el *frame buffer* y el trazador de rayos la escribe en un archivo. Una vez calculada, la imagen se puede seguir procesando sin volver a la escena.

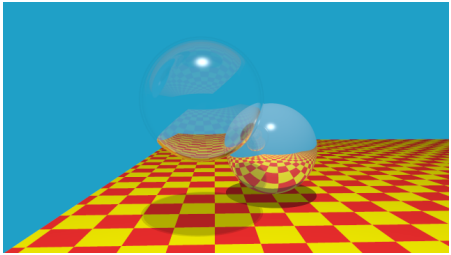


Imagen generada con el trazador de rayos de los talleres

Algunos usos en computación gráfica:

- Corrección gamma y ajustes de brillo y contraste.
- Antialiasing y reconstrucción de la imagen.
- Filtrado de texturas.
- Eliminación del ruido en imágenes generadas con trazado de rayos.

Imágenes en computación gráfica

Contenido de la unidad

1. Representación: la imagen como función, muestreo y cuantización, la imagen como matriz.
2. Operaciones punto a punto y operaciones aritméticas entre imágenes.
3. Filtrado espacial: convolución, suavizado, bordes y filtro de mediana.

Representación de imágenes

La imagen como función

Una imagen se modela como una función de dos variables, $f(x, y)$, donde (x, y) son las coordenadas de un punto del plano y el valor de f es la intensidad (o nivel de gris) en ese punto.

Cuando la imagen proviene de un proceso físico, el valor es proporcional a la energía de la luz que llega a cada punto, por lo que es positivo y finito, $0 < f(x, y) < \infty$. Esa energía se puede separar en dos factores:

$$f(x, y) = i(x, y) \cdot r(x, y)$$

- $i(x, y)$: **iluminación**, la luz que incide en el punto $(0 < i < \infty)$.
- $r(x, y)$: **reflectancia**, la fracción que refleja el objeto $(0 < r < 1)$.

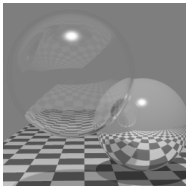
Una imagen en color tiene una función por canal: $f_R(x, y)$, $f_G(x, y)$ y $f_B(x, y)$.

Representación de imágenes

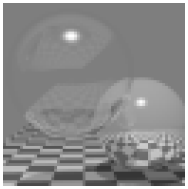
Digitalización: muestreo y cuantización

Para guardar $f(x, y)$ en una computadora hay que discretizar las coordenadas y los valores:

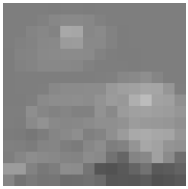
- **Muestreo**: se evalúa f en una grilla de $M \times N$ puntos. Cada muestra es un **píxel** (*picture element*) y $M \times N$ es la **resolución espacial**.
- **Cuantización**: cada valor se redondea a uno de $L = 2^k$ niveles, guardados con k bits. Lo habitual es $k = 8$, es decir, 256 niveles por canal.



256×256



64×64

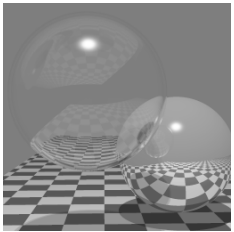


16×16

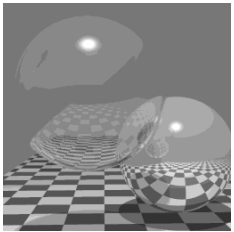
Representación de imágenes

Resolución de intensidad: cantidad de niveles

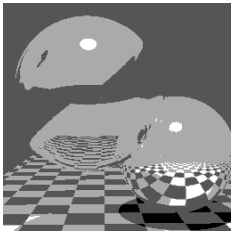
Con pocos niveles de cuantización, una variación suave de intensidad se convierte en bandas de color constante separadas por saltos (*false contouring*).



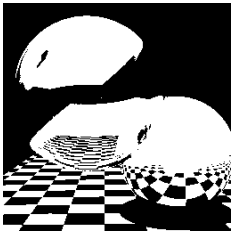
256 niveles ($k = 8$)



16 niveles ($k = 4$)



4 niveles ($k = 2$)



2 niveles ($k = 1$)

Representación de imágenes

La imagen como matriz

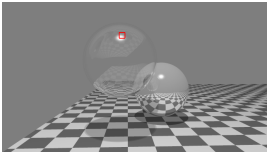
Una imagen digital es una matriz de M filas y N columnas de números enteros. Ampliando un bloque de 8×8 píxeles de la imagen se ven los valores guardados (0 es negro y 255 es blanco).

En memoria, la matriz se guarda por filas en un vector de $M \cdot N \cdot c$ valores, con c canales. El valor del canal k del píxel (x, y) está en la posición

$$(y \cdot N + x) \cdot c + k$$

que es el mismo cálculo que hace cualquier programa que lee una imagen con `stb_image`.

144	144	143	142	141	140	138	137
153	152	151	149	147	145	143	141
167	166	164	161	158	154	150	147
186	184	181	176	171	166	160	155
208	206	201	196	189	181	174	166
231	228	224	217	208	199	189	179
246	245	242	235	226	215	203	191
253	252	250	247	240	229	216	202



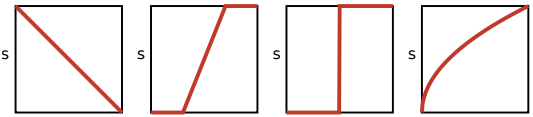
Ubicación del bloque en la imagen (recuadro rojo).

Operaciones punto a punto

Transformaciones de intensidad

En una operación punto a punto, el valor de salida en (x, y) depende sólo del valor de entrada en el mismo píxel. Con $r = f(x, y)$ y $s = g(x, y)$, la operación es una función de una variable, $s = T(r)$, que se aplica a todos los píxeles por igual. Con los valores normalizados entre 0 y 1:

- Negativo: $s = 1 - r$.
- Estiramiento del contraste: $s = \frac{r - r_1}{r_2 - r_1}$, recortado a $[0, 1]$.
- Umbral: $s = 1$ si $r \geq r_0$, y $s = 0$ si no.
- Potencia (gamma): $s = r^\gamma$.

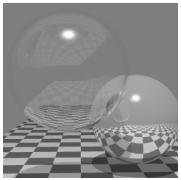


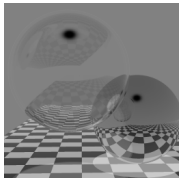
Contraste con $r_1 = 0{,}3$ y $r_2 = 0{,}7$; umbral con $r_0 = 0{,}5$.

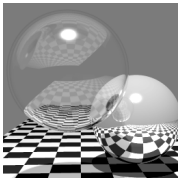
Como T depende sólo de r, con 256 niveles, se puede calcular una vez y guardar en una tabla de 256 entradas (*lookup table*).

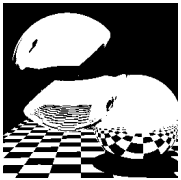
Operaciones punto a punto

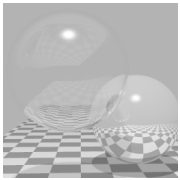
Ejemplos sobre la misma imagen


Original


Negativo


Contraste


Umbral


Gamma 0,5

La corrección gamma de la Unidad III es una operación punto a punto: el trazador de rayos calcula cada canal en valores lineales y al escribir la imagen aplica $s = r^{1/2,2}$.

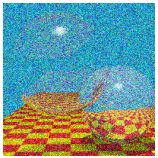
Operaciones aritméticas entre imágenes

Suma: el promedio reduce el ruido

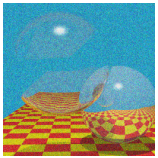
Las operaciones aritméticas se aplican píxel a píxel entre imágenes del mismo tamaño. Si se tienen K imágenes de la misma escena, $g_i = f + \eta_i$, cada una con un ruido η_i de media nula e independiente de los otros, el promedio

$$\bar{g}(x,y) = \frac{1}{K}\sum_{i=1}^{K} g_i(x,y)$$

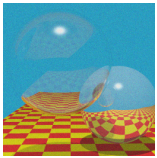
se aproxima a f (sin ruido) y divide la varianza del ruido por K, es decir, la desviación estándar del ruido disminuye con $1/\sqrt{K}$.



$K = 1$



$K = 8$

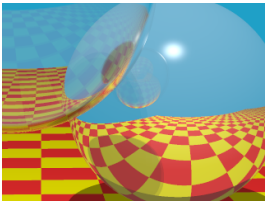


$K = 64$

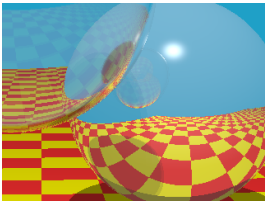
Operaciones aritméticas entre imágenes

Resta: lo que cambia entre dos imágenes

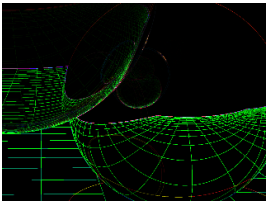
La resta $g = |f_1 - f_2|$ resalta las diferencias entre dos imágenes y deja en negro lo que no cambia. En el ejemplo, las dos imágenes son la misma escena generada con el trazador de rayos, una con supermuestreo de 3×3 rayos por píxel y otra con un solo rayo por píxel.



3×3 rayos por píxel



1 rayo por píxel



Diferencia (amplificada)

Las dos imágenes coinciden en las zonas de color uniforme y difieren sólo en los bordes: el supermuestreo cambia el color de los píxeles que un borde atraviesa.

Filtrado espacial

Operaciones sobre una vecindad

En las operaciones punto a punto cada píxel de salida depende de un solo píxel de entrada. En el **filtrado espacial**, el valor de salida en (x, y) depende de los valores de una **vecindad** de ese píxel, normalmente una ventana de $m \times n$ píxeles centrada en (x, y), con m y n impares (3×3, 5×5, ...).

El filtro se aplica a todos los píxeles: la ventana se desplaza por la imagen y en cada posición se calcula un valor nuevo a partir de los valores que cubre. Hay dos casos:

- **Filtros lineales**: el valor de salida es una suma ponderada de los valores de la vecindad. Los pesos forman una matriz de $m \times n$ que se denomina **kernel** (o máscara, patrón, ventana).
- **Filtros no lineales**: el valor de salida se obtiene con otra operación sobre la vecindad, por ejemplo ordenar los valores y tomar el del medio.

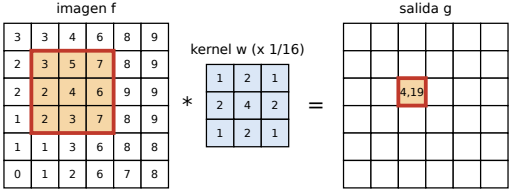
Los valores de salida se escriben en una imagen nueva: si se escribieran sobre la entrada, los píxeles ya filtrados entrarían en el cálculo de sus vecinos.

Filtrado espacial

Convolución discreta

La **convolución** de un filtro lineal, dado por un kernel w de $(2a+1) \times (2b+1)$, con la imagen f es:

$$g(x,y) = (w \star f)(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t)\, f(x-s, y-t)$$



g = (1·7 + 2·3 + 1·2 + 2·6 + 4·4 + 2·2 + 1·7 + 2·5 + 1·3) / 16 = 4,19

Filtrado espacial

Convolución: dos detalles

Correlación y convolución. El signo menos de $f(x-s, y-t)$ hace que el kernel se recorra al revés, lo que equivale a rotarlo 180° antes de superponerlo. La misma sumatoria sin la rotación,

$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t)\, f(x+s, y+t)$$

se denomina **correlación**. Con un kernel simétrico, como el del ejemplo o el de un promedio, las dos dan lo mismo; con uno que no lo es (los de bordes, más adelante) cambia el signo del resultado.

Bordes de la imagen. Cerca del borde, parte de la ventana cae fuera de la imagen y hay que completar los valores que faltan. Las opciones habituales son completar con ceros, repetir el píxel del borde o reflejar la imagen. Con ceros, el borde de la imagen filtrada se oscurece.

Filtrado espacial

Filtros de suavizado (pasa bajo)

Un filtro de suavizado reemplaza cada píxel por un promedio ponderado de su vecindad. Atenúa las variaciones rápidas de intensidad (el ruido y los detalles finos) y conserva las lentas. Los pesos suman 1, por lo que una zona de intensidad constante no cambia.

Filtro de promedio (caja): todos los pesos iguales.

$$w = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Filtro gaussiano: los pesos decrecen con la distancia al centro.

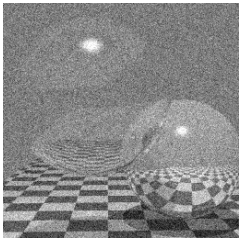
$$w(s,t) = K\, e^{-\frac{s^2+t^2}{2\sigma^2}}$$

K normaliza los pesos para que sumen 1. El kernel se corta en un radio de alrededor de 3σ, porque más lejos los pesos son despreciables.

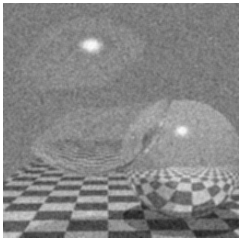
El gaussiano da más peso a los píxeles cercanos y trata igual a todas las direcciones: un punto brillante suavizado con la caja queda como un cuadrado, y con el gaussiano, como un círculo difuso.

Filtrado espacial

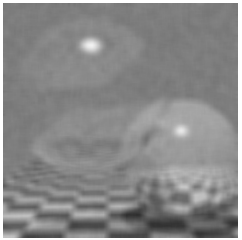
Suavizado de una imagen con ruido



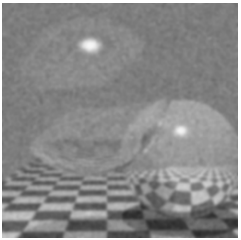
Con ruido gaussiano



Caja de 3×3



Caja de 9×9



Gaussiano, $\sigma = 1{,}5$

Cuanto más grande es el kernel, más se reduce el ruido y más se desdibujan los bordes. El suavizado no distingue el ruido de los detalles de la imagen: los dos son variaciones rápidas de intensidad.

Filtrado espacial

Filtros de bordes (pasa alto)

Un borde es un cambio brusco de intensidad. Los filtros de bordes aproximan derivadas de f con diferencias entre píxeles vecinos, por lo que dan cero en las zonas de intensidad constante y valores grandes en los bordes. Sus pesos suman 0.

Sobel, primera derivada en una dirección por un suavizado en la otra:

$$w_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

$$w_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

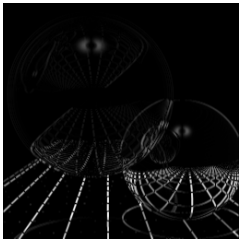
Laplaciano, segunda derivada, igual en todas las direcciones:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad \text{o} \quad \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

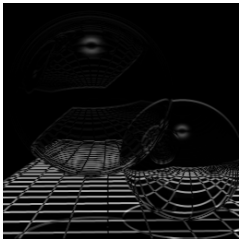
Magnitud del gradiente de Sobel:
$M = \sqrt{g_x^2 + g_y^2}$, que se suele aproximar con $|g_x| + |g_y|$.

Filtrado espacial

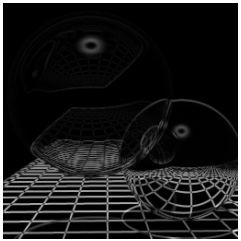
Detección de bordes



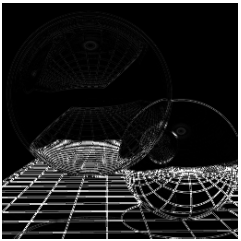
Sobel, $|g_x|$



Sobel, $|g_y|$



Magnitud M



Laplaciano

$|g_x|$ responde a los bordes verticales (la intensidad cambia en x) y $|g_y|$ a los horizontales; la magnitud combina los dos. El Laplaciano no depende de la orientación del borde, pero amplifica más el ruido porque es una segunda derivada.

Filtrado espacial

Filtro de mediana (no lineal)

El filtro de mediana reemplaza cada píxel por la **mediana** de los valores de su vecindad: se ordenan los $m \cdot n$ valores y se toma el del medio. Por ejemplo, en una ventana de 3×3:

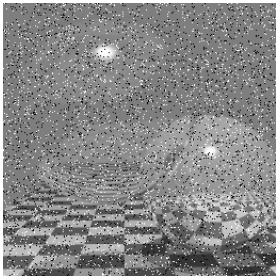
$$\{\,12,\ 14,\ 13,\ 255,\ 15,\ 13,\ 14,\ 12,\ 16\,\} \ \rightarrow\ 12,\ 12,\ 13,\ 13,\ 14,\ 14,\ 15,\ 16,\ 255$$

El promedio de esos nueve valores es 40, mientras que la mediana es 14: un valor extremo aislado no afecta la mediana.

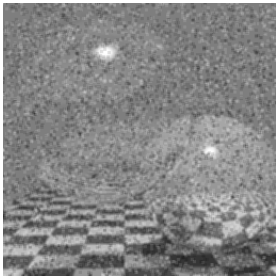
Por eso el filtro de mediana es muy efectivo contra el **ruido impulsivo** (o de sal y pimienta), que pone píxeles aislados en blanco o en negro. Además, la mediana es siempre uno de los valores de la vecindad, por lo que no inventa intensidades intermedias y desdibuja menos los bordes que un promedio.

Filtrado espacial

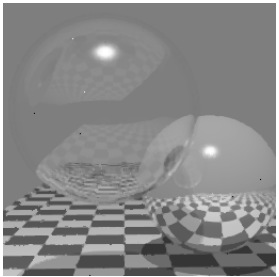
Ruido impulsivo: mediana contra promedio



10 % de píxeles en blanco o negro



Caja de 3×3



Mediana de 3×3

El promedio reparte cada impulso entre sus vecinos y deja manchas grises. La mediana los elimina casi todos y conserva las líneas del tablero.

Filtrado espacial

Costo del filtrado espacial

Una convolución directa con un kernel de $m \times n$ hace $m \cdot n$ multiplicaciones por píxel, es decir, $M \cdot N \cdot m \cdot n$ en total. Para una imagen de 1280×720 y un kernel de 11×11 son unos 111 millones de multiplicaciones por canal.

Algunos kernels son **separables**: se pueden escribir como el producto de un vector columna por un vector fila. La caja y el gaussiano lo son:

$$\frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix} = \frac{1}{3}\begin{bmatrix}1\\ 1\\ 1\end{bmatrix} \cdot \frac{1}{3}\begin{bmatrix}1 & 1 & 1\end{bmatrix}$$

Entonces la convolución 2D se hace con dos convoluciones 1D, una por filas y otra por columnas, con $m + n$ multiplicaciones por píxel en lugar de $m \cdot n$: de 121 a 22 con el kernel de 11×11.

Para kernels grandes y no separables hay otra salida, que es filtrar en el dominio de la frecuencia (bloque 2).

Muestreo y aliasing

El teorema de muestreo

Muestrear es quedarse con los valores de una función continua en puntos separados por una distancia fija Δ, es decir, con una frecuencia de muestreo $f_s = 1/\Delta$. La pregunta es si con esas muestras se puede reconstruir la función original.

Teorema de muestreo (Nyquist-Shannon): una función cuya frecuencia más alta es f_{max} se puede reconstruir exactamente a partir de sus muestras si

$$f_s > 2\, f_{max}$$

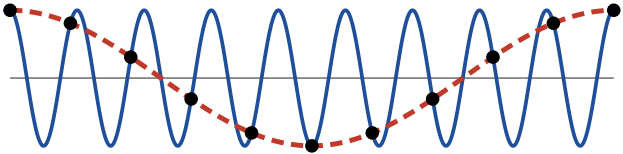
es decir, si se toman más de dos muestras por cada ciclo de la componente más rápida.

Si se muestrea más despacio, las frecuencias altas aparecen en las muestras como frecuencias más bajas que no estaban en la función. Ese fenómeno se denomina **aliasing**.

Muestreo y aliasing

Aliasing en una dimensión

Una senoide de 9 ciclos muestreada con 10 muestras no cumple el teorema, porque necesitaría más de 18. Sus muestras coinciden exactamente con las de una senoide de 1 ciclo, que no está en la señal original:

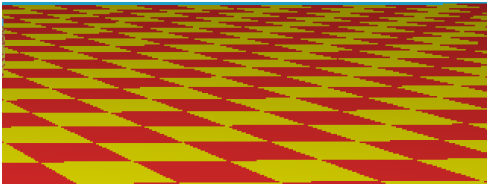


Con las muestras solas no hay forma de distinguir las dos senoides: la reconstrucción da la de 1 ciclo. Una vez que se muestreó, el aliasing ya no se puede corregir.

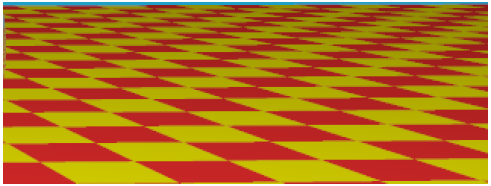
Muestreo y aliasing

Aliasing en imágenes generadas

En una imagen la frecuencia es espacial: qué tan rápido cambia la intensidad de un píxel al siguiente. Un borde neto, como el de un casillero del tablero, contiene frecuencias arbitrariamente altas, por lo que **ninguna** resolución cumple el teorema. Un rayo por píxel es una muestra sin filtrar.



1 rayo por píxel



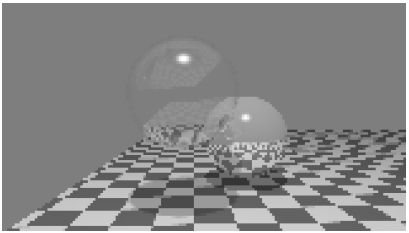
3×3 rayos por píxel

El aliasing aparece como escalones en los bordes (*jaggies*) y, donde los casilleros son más chicos que un píxel, como patrones que no están en la escena (Moiré). El supermuestreo promedia varios rayos por píxel: filtra la escena con un pasa bajo antes de muestrearla.

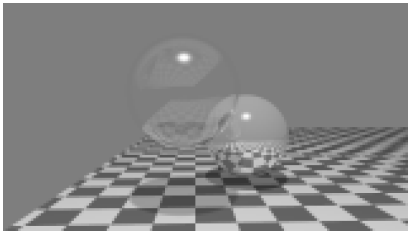
Muestreo y aliasing

Achicar una imagen es volver a muestrearla

Para reducir una imagen a 1/8 de su tamaño se puede tomar un píxel de cada 8 o reemplazar cada bloque de 8×8 por su promedio. El segundo método filtra antes de muestrear y evita el aliasing.



Un píxel de cada 8



Promedio de cada bloque de 8×8

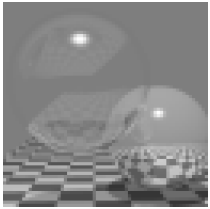
Es lo que resuelven los *mipmaps* en el filtrado de texturas: versiones de la textura ya achicadas con promedio, para no muestrear la textura original cuando se ve lejos.

Muestreo y aliasing

Reconstrucción: interpolar entre muestras

Para ampliar una imagen, o para leer una textura entre los centros de sus píxeles, hay que estimar valores entre las muestras. Esa **reconstrucción** es una interpolación:

- **Vecino más cercano**: el valor del píxel más próximo. Deja ver los píxeles como cuadrados.
- **Bilineal**: los cuatro píxeles más próximos, con pesos según la distancia en x y en y.



64×64 ampliada por vecino más cercano



64×64 ampliada por interpolación bilineal

En OpenGL son los modos de filtrado de las texturas `GL_NEAREST` y `GL_LINEAR`.

Bibliografía recomendada

- GONZALEZ, R. C. & WOODS, R. E.. *Digital Image Processing*. 4ta ed. New York (USA): Pearson, 2018. cap. 2 Digital image fundamentals, p. 47-117; cap. 3 Intensity transformations and spatial filtering, p. 119-201; cap. 4 Filtering in the frequency domain, p. 203-315; sec. 5.3 Restoration in the presence of noise only, p. 327-340.

¿¿¿ PREGUNTAS ???